Redux Toolkit

Introduction

In React applications, managing data across multiple components becomes difficult as the app grows.

For example:

- User authentication data
- Cart items in e-commerce apps
- API data (products, posts)
- Global UI states

Initially, we use **props or Context API**, but for large applications, it becomes hard to manage.

To solve this problem, we use **Redux Toolkit**.

Redux Toolkit is the **modern and recommended way to use Redux**.

---

1. Why Do We Need Redux?

As applications grow:

- Data is needed in many components
- Props become deeply nested (**prop drilling**)
- State becomes hard to manage

Example problem:

```
App
 ↓
Navbar
 ↓
Sidebar
```

```
 ↓
Profile
```

Passing data through all levels becomes messy.

Solution: Redux

Redux provides a **central store** where all data is kept.

```
Redux Store
   ↓
Any Component can access data
```

---

2. What is Redux?

Redux is a **state management library**.

It stores all application data in a **single global store**.

Basic idea:

```
Component
   ↓
Dispatch Action
   ↓
Reducer updates state
   ↓
Store updates
   ↓
Component re-renders
```

---

3. What is Redux Toolkit?

Redux Toolkit (RTK) is the **official, simplified way to use Redux**.

It removes:

- Boilerplate code

- Complex setup
- Manual reducer logic

RTK provides:

- `configureStore`
- `createSlice`
- Built-in middleware
- Better developer experience

---

4. Context API vs Redux

Feature	Context API	Redux Toolkit
Use case	Small apps	Medium/Large apps
Performance	Can re-render often	Optimized
DevTools	No	Yes
Structure	Simple	Structured
Async handling	Manual	Built-in (thunk)

When to use:

- Context → Theme, auth (small apps)
- Redux → Complex state, large apps

---

5. Core Concepts of Redux

```
Redux
 ├── Store
 ├── Slice
 ├── Reducer
 ├── Action
 ├── Dispatch
 └── Selector
```

Store

The **store** holds the entire application state.

Example:

```
import { configureStore } from "@reduxjs/toolkit";
import userReducer from "./userSlice";

export const store = configureStore({
  reducer: {
    user: userReducer
  }
});
```

👉 Here, `store` contains all app data (like users, auth, cart, etc.)

Slice

A slice is a **part of the state + logic together**.

Example:

```
import { createSlice } from "@reduxjs/toolkit";

const userSlice = createSlice({
  name: "user",
  initialState: {
    users: []
  },
  reducers: {
    setUsers: (state, action) => {
      state.users = action.payload;
    }
  }
});
```

```
export const { setUsers } = userSlice.actions;
export default userSlice.reducer;
```

👉 Slice = State + Reducers + Actions in one place

Reducer

A reducer updates the state based on an action.

Example:

```
setUsers: (state, action) => {
  state.users = action.payload;
}
```

👉 When `setUsers` is called, it updates the state.

Action

An action describes **what should happen**.

Example:

```
setUsers([{ id: 1, name: "Ritik" }])
```

👉 This action tells Redux:

> "Update users with this data"

Dispatch

Dispatch is used to **trigger actions**.

Example:

```
import { useDispatch } from "react-redux";
import { setUsers } from "./userSlice";

const dispatch = useDispatch();

dispatch(setUsers([{ id: 1, name: "Ritik" }]));
```

👉 Dispatch sends the action to Redux.

Selector

Selector is used to **read data from the store**.

Example:

```
import { useSelector } from "react-redux";

const users = useSelector((state) => state.user.users);

console.log(users);
```

👉 It gets data from the Redux store.

Complete Flow Example

```
// Dispatch
dispatch(setUsers([{ id: 1, name: "Ritik" }]));
```

```
Dispatch Action
↓
Reducer runs
↓
Store updates
↓
Component re-renders
```

👉 This is how Redux works internally.

6. Installing Redux Toolkit

```
npm install @reduxjs/toolkit react-redux
```

7. Setting Up Redux

Step 1: Create Store

redux/store.js

```
import { configureStore } from "@reduxjs/toolkit";
import userReducer from "./slices/userSlice";

export const store = configureStore({
  reducer: {
    user: userReducer
  }
});
```

Step 2: Create Slice

redux/slices/userSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  users: []
};
```

```
const userSlice = createSlice({
  name: "user",
  initialState,
  reducers: {
    setUsers: (state, action) => {
      state.users = action.payload;
    }
  }
});

export const { setUsers } = userSlice.actions;
export default userSlice.reducer;
```

Step 3: Connect Redux to React

main.jsx

```
import { Provider } from "react-redux";
import { store } from "./redux/store";

<Provider store={store}>
  <App />
</Provider>
```

8. Using Redux in Components

Reading Data (useSelector)

```
import { useSelector } from "react-redux";

const users = useSelector(state => state.user.users);
```

Updating Data (useDispatch)

```
import { useDispatch } from "react-redux";
import { setUsers } from "../redux/slices/userSlice";

const dispatch = useDispatch();

dispatch(setUsers(data));
```

---

9. Four Layered Architecture (Industry Standard)

```
src/
 ├── components/   (UI Layer)
 ├── hooks/        (Container Layer)
 ├── redux/        (Redux Layer)
 ├── services/     (Service Layer)
```

Layer Explanation

1. UI Layer (components)

- Only UI
- No business logic

---

2. Container Layer (hooks)

- Handles logic
- Calls APIs
- Dispatches Redux actions

---

3. Redux Layer

- Store
- Slices
- Reducers

---

4. Service Layer

- API calls (Axios)

---

10. API Integration Example

Backend Example (Node.js)

```
// server.js
const express = require("express");
const app = express();

app.get("/api/users", (req, res) => {
  res.json([
    { id: 1, name: "Ritik" },
    { id: 2, name: "John" }
  ]);
});

app.listen(5000);
```

Service Layer

services/userService.js

```
import axios from "axios";

export const fetchUsersAPI = () => {
```

```
  return axios.get("http://localhost:5000/api/users");
};
```

Hook (Container Layer)

hooks/useUsers.js

```
import { useEffect } from "react";
import { useDispatch, useSelector } from "react-redux";
import { fetchUsers } from "../redux/slices/userSlice";

export const useUsers = () => {
  const dispatch = useDispatch();

  const { data, loading, error } = useSelector((state) => sta
te.users);

  useEffect(() => {
    dispatch(fetchUsers());
  }, [dispatch]);

  return { data, loading, error };
};
```

Component (UI Layer)

components/Users.jsx

```
import { useSelector } from "react-redux";
import { useUsers } from "../hooks/useUsers";

function Users() {
```

```
  useUsers();

  const users = useSelector(state => state.user.users);

  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default Users;
```

11. Data Flow in Redux Architecture

```
Component
   ↓
Custom Hook (useUsers)
   ↓
Service Layer (API call)
   ↓
Dispatch Action
   ↓
Slice Reducer updates state
   ↓
Store updates
   ↓
Component re-renders
```

12. Async Handling (Redux Toolkit)

Redux Toolkit provides **createAsyncThunk**.

Example:

```
import { createAsyncThunk, createSlice } from "@reduxjs/toolk
it";
import axios from "axios";

export const fetchUsers = createAsyncThunk(
  "users/fetchUsers",
  async () => {
    const res = await axios.get("/api/users");
    return res.data;
  }
);

const userSlice = createSlice({
  name: "user",
  initialState: { users: [], loading: false },
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.users = action.payload;
        state.loading = false;
      });
  }
});

export default userSlice.reducer;
```

13. Advantages of Redux Toolkit

- Less boilerplate
- Better performance

- Centralized state
- DevTools support
- Easy async handling

---

14. When to Use Redux

Use Redux when:

- App is large
- Many components share data
- Complex state logic
- API-heavy applications

Conclusion

Redux Toolkit helps manage **global state efficiently in large applications**.

Important concepts:

```
Store        → Global state
Slice        → State + logic
Reducer      → Updates state
Action       → What to do
Dispatch     → Triggers action
Selector     → Reads state
```

Architecture:

```
UI → Hooks → Services → Redux → Store
```

Redux Toolkit makes your app:

- Scalable
- Maintainable

- Industry-ready